

Programare paralelă și distribuită în MPI.

Obiectivele lucrării

Lucrarea își propune studierea și însușirea noțiunilor teoretice precum și rezolvarea unor probleme concrete legate de lucrul cu standardul de comunicație MPI, urmărind familiarizarea cu aspectele importante specifice programării paralele și distribuite bazată pe transferul de mesaje. Sarcinile laboratorului includ:

- ⇒ Exercițiul 1 – Operații colective în MPI.
- ⇒ Exercițiul 2 – Tipuri de date în MPI.
- ⇒ Exercițiul 3 – Topologii (virtuale) de procese în MPI.
- ⇒ Exercițiul 4 – Aplicații.

Exercițiul 1 – Operații colective în MPI

Operațiile colective implică toate procesele din cadrul unui comunicator. Toate procesele sunt membre ale comunicatorului inițial, predefinit MPI_COMM_WORLD.

Tipuri de operații colective:

- **Sincronizare:** procesele așteaptă toți membrii grupului să ajungă în punctul de joncțiune.
- **Transfer de date** - broadcast, scatter/gather, etc.
- **Calcule colective** (reductions) – un membru al grupului colectează datele de la toți ceilalți membrii și realizează o operație asupra acestora (minim, maxim, adunare, înmulțire, etc.)

Sincronizarea la barieră este cea mai simplă operație de comunicare colectivă care nu implică transfer de date. Forma sa este: `MPI_BARRIER (MPI_comm comm)`.

Unicul argument este comunicatorul *comm* ce definește grupul de procese sincronizate, astfel procesul apelant se blochează până când toți

Transmisia broadcast.

Forma funcției este:

```
int MPI_Bcast (void *buffer, int count, MPI_datatype  
datatype, int source, MPI_comm comm)
```

unde *buffer* este adresa de început a buffer-ului, *count* este numărul intrărilor în *buffer*, *datatype* este tipul datelor din *buffer*, *source* (poate fi cazul particular *root*) este rangul procesului sursă, iar *comm* este comunicatorul.

Un exemplu de utilizare a funcției *MPI_Bcast* este prezentat în figura 2.1.

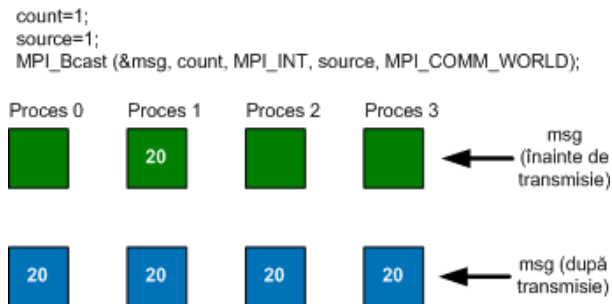


Figura 2.1 Utilizare funcție *MPI_Bcast*

Distribuirea datelor de la un proces la toate celelalte procese din comunicator (MPI_Scatter, MPI_Scatterv). Sintaxa funcției MPI_Scatter este:

```
int MPI_SCATTER (void *sendbuf, int sendcount,
MPI_Datatype sendtype, void recvbuf, int recvcount,
MPI_Datatype recvtype, int root, MPI_comm comm).
```

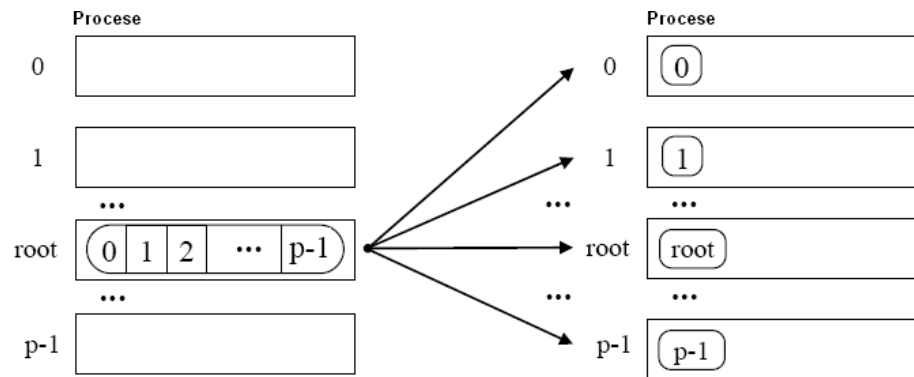


Figura 2.2 Schema generală de transmitere a datelor de la un proces la toate celelalte prin funcția MPI_Scatter

Când este apelată această funcție, procesul cu rangul *root* transmite datele sale la toate celelalte procese din comunicator. *Sendcount* elemente vor fi trimise fiecărui proces. Procesul 0 va recepționa blocul de date din *sendbuf* în care elementele au indicii de la 0 la *sendcount-1*, procesul cu rangul 1 va recepționa datele cu indicii de la *sendcount* la *2*sendcount-1*, ș.a.m.d. Astfel, dimensiunea mesajului trebuie să fie egală cu *sendcount*p*, unde *p* reprezintă numărul de procese din comunicator.

În situația în care un număr diferit de elemente din mesaj trebuie trimis fiecărui proces din comunicator se poate folosi funcția *MPI_Scatterv*.

Colectarea datelor de la toate procesele din comunicator la un singur proces (MPI_Gather, MPI_Gatherv).

Cele *n* mesaje sunt concatenate în ordinea rangurilor proceselor și stocate în buferul de recepție al procesului receptor (figura 2.3).

Fiecare bufer imitator este identificat de tripletul *sendbuf*, *sendcount*, *sendtype*, iar buferul de recepție este ignorat pentru procesele *nonroot*, iar pentru procesul *root* este identificat de tripletul *recvbuf*, *recvcount*, *recvtype*. Sintaxa funcției *MPI_Gather* este: `int MPI_GATHER (void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_comm comm).`

unde:

- *sendbuf* este adresa de start a send buffer;
- *sendcount* este numărul elementelor din send buffer;
- *sendtype* este tipul datelor din send buffer;
- *recvbuf* este adresa lui receive buffer;
- *recvcount* este numărul elementelor pentru fiecare recepție;
- *root* este rangul procesului receptor.

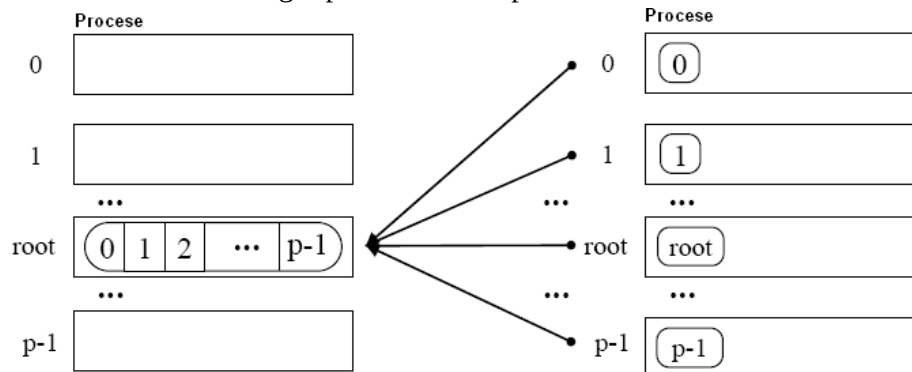


Figura 2.3 Schema generală de colectare a datelor de la toate procesele din comunicator prin funcția *MPI_Gather*

Funcția implementează comunicația de tip *all to one*, toate procesele transferă datele aflate în *sendbuf* procesului rădăcină, care plasează datele în locații contigue nesuprapuse în buferul *recvbuf*.

În situația în care un număr diferit de elemente din mesaj trebuie colectat de la fiecare proces din comunicator se poate folosi funcția *MPI_Gatherv*.

În situația în care se dorește concatenarea de date (de) la toate procesele din comunicator se folosește funcția *MPI_AllGather*, respectiv *MPI_Allgatherv*.

```
int MPI_AllGather ( void *sendbuf, int sendcount,
MPI_Datatype sendtype, void *recvbuf, int recvcount,
MPI_Datatype recvtype, MPI_Comm comm ).
```

Pentru această funcție nu se specifică un proces *root*, toate procesele din comunicator primesc rezultatele. Ca efect, fiecare proces din grup execută o operație de difuzare unu-la-toate, în cadrul grupului.

Operația de schimb total (MPI_Alltoall, MPI_Alltoallv). Operația de schimb total presupune ca fiecare proces să transmită un mesaj personalizat fiecărui proces din grupul de n procese, inclusiv lui însuși. Cele n mesaje sunt stocate în ordinea rangurilor în fiecare buffer de emisie, astfel fiecare proces primește un mesaj de la toate celelalte procese. Sintaxa funcției este:

```
int MPI_Alltoall ( void *sendbuf, int sendcount,
MPI_Datatype sendtype, void *recvbuf, int recvcount,
MPI_Datatype recvtype, MPI_comm comm ) .
```

Cele n mesaje sunt concatenate în ordinea rangurilor și stocate în buferul de recepție. Această operație este echivalentul a n operații *gather*, fiecare generată de un proces diferit, astfel argumentul *root* nu mai este necesar. Parametrii *sendcount* și *recvcount* definesc numărul de elemente transmise și recepționate de fiecare proces individual.

Operații de reducere (MPI_Reduce, MPI_Allreduce). Operațiile de reducere sunt utilizate când dorim să obținem un rezultat ce impune folosirea datelor tuturor proceselor (figura 2.4).

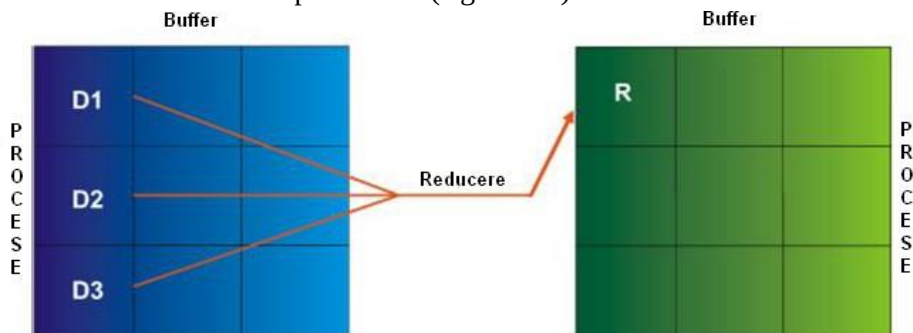


Figura 2.5 Schema generală pentru *MPI_Reduce*

Standardul propune un set de operații predefinite, însă utilizatorul are și posibilitatea definirii altor operații. Operatorii de reducere MPI predefiniți sunt prezentate în tabelul 2.1.

Tabelul 2.1 Operatori MPI predefiniți

Nr. Crt.	Operația de reducere MPI	
1	MPI_MAX	Maxim
2	MPI_MIN	Minim
3	MPI_SUM	Suma
4	MPI_PROD	Produs
5	MPI LAND	AND logic
6	MPI_BAND	AND pe biți

7	MPI_LOR	OR logic
8	MPI_BOR	OR pe biți
9	MPI_LXOR	XOR logic
10	MPI_BXOR	XOR pe biți
11	MPI_MAXLOC	Maxim și locația
12	MPI_MINOC	Minim și locația

Toate operațiile predefinite sunt asociative și comutative. Operația de reducere presupune comunicație de tip *all to one* între procesele integrate în comunicatorul *comm*.

```
int MPI_Reduce (void *sendbuf, void*recvbuf, int count,
MPI_Datatype datatype, MPI_op op, int root, MPI_comm comm) .
```

Funcția combină elementele stocate în buferele *sendbuf* ale fiecărui proces din grup folosind operatorul *op* și întoarce rezultatul combinării în *recvbuf* al procesului *root*. Toate procesele trebuie să ofere un tablou *recvbuf* chiar dacă nu sunt “ținta” operației de reducere. Toate procesele implicate în comunicație trebuie să apeleze funcția cu aceleași valori pentru *count*, *datatype*, *op*, *root* și *comm*.

În figura 2.5 prezentăm rezultatul obținut în cazul în care se utilizează funcția *MPI_Reduce* pentru 5 elemente și *root* 0.

MPI_Reduce (sendbuf, recvbuf, 5, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD).

Proces 0 sendbuf		Proces 1 sendbuf		Proces 2 sendbuf		Proces 3 sendbuf		Proces 0 recvbuf
1		2		3		4		10
2		3		4		5		14
3	+	4	+	5	+	6	=	18
4		5		6		7		22
5		6		7		8		26

Figura 2.5 Schema generală pentru *MPI_Reduce* cu 5 elemente și 4 procese

Dacă rezultatul operației de reducere este necesar tuturor proceselor atunci se utilizează funcția *MPI_Allreduce*.

```
int MPI_Allreduce (void *sendbuf, void*recvbuf, int count,
MPI_Datatype datatype, MPI_op op, MPI_comm comm) .
```

De remarcat că această funcție nu include ca parametru un proces țintă, deoarece rezultatul operațiilor este recepționat de toate procesele din comunicator. Această funcție este echivalentă cu un *MPI_Reduce* urmată de un *MPI_Bcast*.

Exercițiul 2 – Tipuri de date în MPI

MPI a fost proiectat pentru a opera în medii neomogene, fapt ce a impus introducerea unui *mod propriu de reprezentare a datelor*. Tipurile de date pot fi primitive (numere întregi, reale) sau definite de utilizator. O caracteristică deosebit de puternică a bibliotecii o constituie posibilitatea introducerii de argumente sub forma unor tipuri de date diferite, pentru toate mesajele trimise și recepționate. MPI asigură un set deosebit de complex de tipuri de date predefinite, set care include toate tipurile de bază din C, C++ și Fortran, alături de două tipuri de date specifice MPI: *MPI_BYTE* și *MPI_PACKED*. Tipurile de date încep cu *MPI_*, iar corespondența lor cu cele din limbajul C este prezentată în tabelul 2.2.

Tabelul 2.2 *Tipuri de bază MPI predefinite pentru C, C++*

Nr. Crt.	Tipuri MPI	Tipuri C, C++
1	MPI_BYTE	Byte
2	MPI_CHAR	Signed char
3	MPI_DOUBLE	Double
4	MPI_FLOAT	Float
5	MPI_INT	Signed int
6	MPI_LONG	Signed long int
7	MPI_LONG_DOUBLE	Long double
8	MPI_SHORT	Signed short int
9	MPI_UNSIGNED_CHAR	Unsigned char
10	MPI_UNSIGNED	Unsigned int
11	MPI_UNSIGNED_LONG	Unsigned long int
12	MPI_UNSIGNED_SHORT	Unsigned short int
13	MPI_LONG_LONG_INT	Long long int
14	MPI_PACKED	Nu are corespondent în C
15	MPI_FLOAT_INT	Float and int 1
16	MPI_DOUBLE_INT	Double and int 1
17	MPI_LONG_DOUBLE_INT	Long double and int 1
18	MPI_LONG_INT	Long and int 1
19	MPI_SHORT_INT	Short and int 1

Corespondența tipurilor și conversiile datelor se referă la corespondența tipurilor limbajului gazdă (C, C++) cu tipurile specificate în operațiile de comunicare, respectiv corespondența tipurilor la transmițător și

receptor. De notat că folosirea standardului MPI în medii eterogene necesită conversii de tip.

MPI definește anumite tipuri de date abstracte, conversia între diferitele moduri de reprezentare ale acestora pe arhitecturi diverse, fiind realizată **implicit** de subnivelul de comunicație MPI. Pe lângă setul datelor fundamentale (prezentate în tabelul 2.2), MPI oferă mecanisme pentru construcția de *tipuri de date agregate*, ca structuri de tip vectori. Noul tip de date se construiește ca o listă de perechi (tip date, deplasament) și poate fi folosit asemeni unui tip fundamental în MPI. Astfel, standardul MPI permite definirea de tipuri de date derivate, sub forma unui nou obiect specificat ca o secvență de tipuri de bază și respectiv deplasamentele acestora, necesar aplicațiilor de o complexitate ridicată. Un tip de date derivat poate fi văzut ca un *șablon* care se suprapune peste memorie.

În multe situații, datele care trebuie transmise nu se află într-o zonă continuă de memorie și nu pot fi transmise într-un singur mesaj folosind un tip de bază.

Dacă este necesară *trimiterea de date neînvecinate în memorie*, dar **de același tip**, programatorul poate opta pentru una din soluțiile:

- să efectueze apeluri repetate de funcții MPI pentru a trimite și recepționa fiecare dată în parte, soluție care este *lentă și greoaie*.
- să copieze datele pregătite într-un buffer înainte de a le trimite, lucru care ar atrage consum de memorie.

Dacă este necesară *trimiterea de date învecinate în memorie*, dar **de tipuri diferite**, programatorul ar putea opta pentru una din soluțiile:

- să efectueze apeluri repetate de funcții MPI pentru a trimite și recepționa fiecare dată în parte, soluție care este *lentă și greoaie*.
- să folosească *MPI_BYTE* și *sizeof* putând evita regulile de matching pentru tipurile de date, fapt ce ar conduce la un program MPI care să nu fie portabil pe mașini eterogene. Datele neînvecinate de diferite tipuri ridică ambele probleme întâlnite mai sus.

Caracteristicile tipurilor de date derivate sunt:

- **harta tipului** = secvență de perechi (*tip, deplasare*), *Typemap* = $\{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1})\}$.
- **semnătura tipului** = secvența tipurilor, *Typesig* = $\{type_0, \dots, type_{n-1}\}$.
- **titlu** (handle) asociat unui tip general, folosit în operațiile de transmitere/recepție, pentru a specifica tipul datelor comunicate. Tipurile de bază sunt cazuri particulare, predefinite.
- **extindere** (extent) = spațiul dintre primul și ultimul octet ocupat de intrările tipului, rotunjit superior din motive de aliniere.

Exemplu:

Typemap = {(double, 0), (char, 8)} are extinderea 16, dacă valorile *double* trebuie aliniate la adrese multiplu de 8.

Extinderea și dimensiunea unui tip de date pot fi aflate prin următoarele funcții:

```
int MPI_TYPE_EXTENT (MPI_Datatype datatype, MPI_Aint *extent), unde MPI_Aint este un tip întreg care poate reprezenta o adresă oarecare (de obicei este int).
```

```
int MPI_TYPE_SIZE (MPI_Datatype datatype, int * size)
```

MPI oferă o soluție portabilă și eficientă de a comunica datele neînvecinate și/sau de diferite tipuri într-un mesaj. Tipurile de date derivate sunt create în timpul execuției programului. Acest lucru presupune construcția tipului de date. Definirea noilor tipuri de date din cele existente deja (de bază sau derivate) se face apelând una din următoarele rutine:

- MPI_TYPE_CONTIGUOUS,
- MPI_TYPE_VECTOR,
- MPI_TYPE_HVECTOR,
- MPI_TYPE_INDEXED,
- MPI_TYPE_HINDEXED,
- MPI_TYPE_STRUCT.

În continuare vor fi prezentate doar rutinele MPI_TYPE_VECTOR și MPI_TYPE_STRUCT, celelalte fiind similare.

```
int MPI_TYPE_VECTOR (int count, int blocklength, int step, MPI_Datatype oldtype, MPI_Datatype *newtype)
```

Noul tip de date construit constă dintr-un număr de *count* blocuri unde fiecare bloc este format dintr-un număr de *blocklength* copii ale vechiului tip. Elementele din fiecare bloc au deplasamente învecinate, dar deplasamentele blocurilor diferă prin pasul *step*. Acesta permite specificarea unor date situate în zone necontigue de memorie. Elementele tipului vechi pot fi separate între ele de spații având lungimea egală cu un multiplu al extinderii tipului (deci cu un pas constant).

```
int MPI_TYPE_STRUCT (int count, int array_of_blocklengths[], MPI_Aint array_of_displacements[], MPI_Datatype * array_of_types[], MPI_Datatype* newtype)
```

Tipul *struct* este o generalizare a tipului precedent, prin aceea că permite ca fiecare bloc să se constituie din replici ale unor tipuri de date diferite. Noul tip de date constă dintr-o listă de *count* blocuri, unde fiecare bloc din listă este compus dintr-un număr de *array_of_blocklengths[i]* copii ale tipului *array_of_types[i]*. Deplasamentul fiecărui bloc este exprimat în bytes și este dat de *array_of_displacements[i]*.

Operațiile *send* și *receive* au fost realizate corect dacă mapările de tipuri (*type_maps*) pentru tipurile de date specificate, cu deplasamentele

ignore, se potrivesc conform regulilor de matching pentru tipurile de date de bază. Numărul de elemente de bază primite se poate afla folosind rutina *MPI_GET_ELEMENTS*. Rutina *MPI_GET_COUNT* întoarce numărul de elemente primite de tipul specificat în apelul funcției de primire, iar dacă acesta nu este un număr întreg va returna *MPI_UNDEFINED*.

Exercițiul 4 – Aplicații

1. Un exemplu de program MPI în care folosim o transmisie de tip „broadcast” și o recepție non-blocantă. Programul constă în trimiterea unui mesaj de către procesul master tuturor proceselor implicate în comunicare (trebuie să avem minim 2 și maxim 8). Procesele slave recepționează mesajul ce conține numele calculatorului pe care rulează. Programul poate fi gândit ca un test pentru a vedea care dintre procese este mai rapid în retransmiterea mesajului spre master.

```
#include <mpi.h>
#include <stdio.h>
#include "mpi.h"
#include <winsock2.h>

#define MAXPROC 8 //numărul maxim de procese
#define NAMELEN 80 //lung. max. admisă pentru nume calculator

int main(int argc, char* argv[])
{
    int i, np, me, index;
    const int tag = 42; /* eticheta pentru comunicare */
    const int root = 0; /* procesul cu nr 0 pt broadcast */

    MPI_Status status; /* obiectul status pentru recepție
non-blocantă */
    MPI_Request recv_req[MAXPROC]; //obiecte necesare pentru
recepție corectă

    char myname[NAMELEN]; /* memorare nume calculator de
pe care s-a lansat programul*/
    char hostname[MAXPROC][NAMELEN]; //memorare nume calculat.
    MPI_Init(&argc, &argv); /* initializare MPI */
    MPI_Comm_size(MPI_COMM_WORLD, &np); /* nr. de procese */
```

```
MPI_Comm_rank(MPI_COMM_WORLD, &me); //identificarorul pt
fiecare proces
gethostname(myname, NAMELEN); /* nume calculator */
if (me == 0) { /* Procesul 0 execută acest cod*/
// Mai întâi verificăm dacă avem cel puțin 2 și cel mult
MAXPROC procese
if (np<2 || np>MAXPROC) {
printf("Trebuie minim 2 si maxim 8procese\n",
MAXPROC);
MPI_Finalize();
exit(0);
}

printf("Procesul %d de pe calculatorul %s broadcast la
toate celelalte procese\n",me, myname);
/* mesaj ce conține id-ul procesului */
MPI_Bcast(&me, 1, MPI_INT, root, MPI_COMM_WORLD);
printf("Receptie mesaje de la celelalteprocese\n");
/* Receptie mesaje de la celelalte procese */
for (i=1; i<np; i++) {
MPI_Irecv (&hostname[i], NAMELEN, MPI_CHAR, i, tag,
MPI_COMM_WORLD, &recv_req[i]);
}
/* în timp ce este trimis mesajul putem face aici alte
calcule*/
for (i=1; i<np; i++) {
/*Așteaptă până când a recepționat cel puțin un mesaj începe
de la 1 pt că nu se recepționează nimic de la proces 0*/
MPI_Waitany(np-1, &recv_req[1], &index, &status);
/* Afișează de la cine a primit mesaj */
printf("Mesaj primit de la procesul %d de pe calc. %s
\n", status.MPI_SOURCE, hostname[status.MPI_SOURCE]);
}
printf("Gata\n");
} else { /* toate celelalte procese */
int y; /* Variabilă locală pentru broadcast */
/* Verificare corectitudine */
if (np<2 || np>MAXPROC) {
MPI_Finalize();
exit(0);
}
/* Receptie mesaje de la procesul 0 */
MPI_Bcast(&y, 1, MPI_INT, root, MPI_COMM_WORLD);
/* Trimite identificatorul său înapoi procesului 0 */
MPI_Send (&myname, NAMELEN, MPI_CHAR, 0, tag,
MPI_COMM_WORLD);
}
MPI_Finalize();
}
```

2. Un exemplu de program MPI care utilizează procedura MPI_Gather.

Programul trebuie rulat utilizând un număr par de procese și nu mai mare de 8. Fiecare proces inițializează o matrice x de întregi. Aceste matrice sunt colectate de procesul 0 utilizând funcția MPI_Gather și memorate în matricea y . Apoi, toate procesele, cu excepția lui 0, trimit un mesaj ce conține numele mașinii pe care rulează și partea sa de elemente din matrice. Procesul 0 recepționează și afișează mesajele primite de la celelalte procese.

```
#include <stdio.h>
#include "mpi.h"
#include <winsock2.h>

#define MAXPROC 8    /* numărul maxim de procese */
#define NAMELEN 80   /*lungimea maximă admisă pentru nume calc.*/
#define LENGTH 24    /* lungimea buffer-ului divizibilă 2, 4,
6 și 8 */

void main(int argc, char* argv[]) {
    int i, j, np, me;
    const int nametag = 42; /* eticheta pt. trimitere nume */
    const int datatag = 43; /* eticheta pt. trimitere date */
    const int root = 0; /* procesul 0 */
    MPI_Status status; /* starea unei recepții */

    char myname[NAMELEN]; /* memorare host name */
    char hostname[MAXPROC][NAMELEN]; /* receptie host names */

    int x[LENGTH]; /* buffer transmis */
    int y[LENGTH]; /* buffer de recepție*/

    MPI_Init(&argc, &argv); /* initializare MPI */
    MPI_Comm_size(MPI_COMM_WORLD, &np); /* nr de procese */
    MPI_Comm_rank(MPI_COMM_WORLD, &me); /*identificatorul
    fiec. proces */
    gethostname(myname, NAMELEN); /* obținere host name */

    // fiecare proces inițializează propriamatrice
    for (i=0; i<LENGTH/np; i++)
        x[i] = (LENGTH/np)*me+i;
    if (me == 0) { /* Proces 0 execută */
        /* verificam daca avem un număr par de procese și maxim
        MAXPROC */
        if (np>MAXPROC || np%2 != 0) {
            printf("Trebuie sa utilizati un numar par de procese
            (cel mult %d)\n", MAXPROC);
            MPI_Finalize();
            exit(0);
        }
    }
}
```

```
printf("Procesul %d de pe calculatorul radacina %s
colecteaza datele de la cele %d procese\n\n", me, myname,
np);
/* trecere (grupare) matrice x in matrice y */
MPI_Gather(&x, LENGTH/np, MPI_INT, &y, LENGTH/np,
MPI_INT, root, MPI_COMM_WORLD);
/* afisare matrice y */
printf("Procesul %d de pe calculatorul %s are
elementele\n", me, myname);
for (i=0; i<LENGTH; i++)
printf(" %d", y[i]);
printf("\n\n");
/* afisare matrice x construită de procesul 0 */
printf("Procesul %d de pe calculatorul %s are
elementele", me, myname);
for (i=0; i<LENGTH/np; i++)
printf(" %d", x[i]);
printf("\n");
//receptie mesaje ce conțin nume calculator și matricea
coresp. fiecărui proces
for (i=1; i<np; i++) {
MPI_Recv (&hostname[i], NAMELEN, MPI_CHAR, i, nametag,
MPI_COMM_WORLD, &status);
MPI_Recv (&y, LENGTH/np, MPI_INT, i, datatag,
MPI_COMM_WORLD, &status);
printf("Procesul %d de pe calculatorul %s are
elementele", i, hostname[i]);
for (j=0; j<LENGTH/np; j++)
printf(" %d", y[j]);
printf("\n");
}
printf("Finalizare\n");
}
else { /* execută procesele cu me!=0 */
if (np>MAXPROC || np%2 != 0) {
MPI_Finalize();
exit(0);
}
MPI_Gather(&x, LENGTH/np, MPI_INT, &y, LENGTH/np,
MPI_INT, root, MPI_COMM_WORLD);
/* trimite, numele calculatorului pe care rulează,
procesului 0 */
MPI_Send (&myname, NAMELEN, MPI_CHAR, 0, nametag,
MPI_COMM_WORLD);
/* se trimite matricea procesului 0 */
MPI_Send (&x, LENGTH/np, MPI_INT, 0, datatag,
MPI_COMM_WORLD);
}
MPI_Finalize();
}
```

Pentru 4 procese și dimensiune matrice 24 avem rezultatele prezentate în figura 2.7.

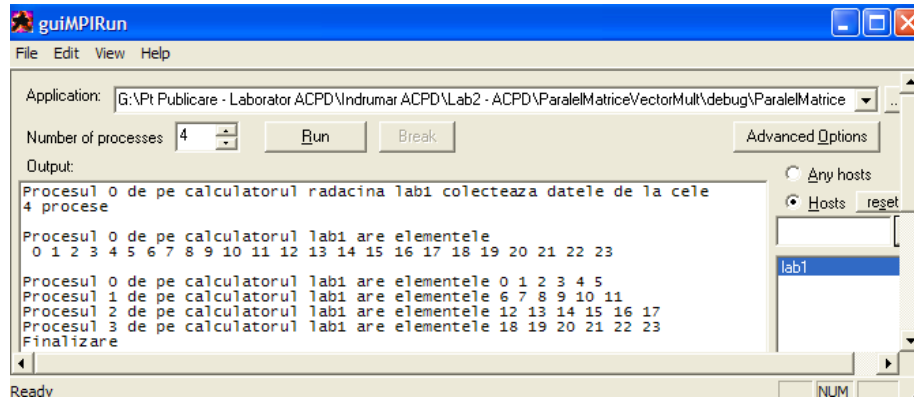


Figura 2.7 Verificare colectare corectă date pentru 4 procese și dimensiune matrice egală cu 24

3. Prezentăm un exemplu de program MPI ce implementează operația de distribuire a datelor către procesoare utilizând funcția `MPI_Scatter`. Pentru rulare aplicația necesită 8 procese. Procesul 0 inițializează o matrice de 8×8 numere întregi și distribuie o parte din aceste elemente către procesele implicate în comunicație. Distribuția se face astfel: procesul 0 primește un element, procesul 1 primește 2 elemente, ... , procesul i primește $i+1$ elemente. Toate procesele cu excepția lui 0 retrimite mesaj înapoi cu numele mașinii pe care rulează și elementele recepționate.

```
#include <stdio.h>
#include "mpi.h"
#include <winsock2.h>
#define MAXPROC 8 /* Nr.max de procese */
#define NAMELEN 80 /* Lungimea max. a numelui calc. */
#define LENGTH 8 /* Mărimea matricei LENGTH * LENGTH */

void main(int argc, char* argv[]) {
    int i, j, np, me, count;
    const int nametag = 42;
    const int datatag = 43;
    const int root = 0;
    MPI_Status status;

    char myname[NAMELEN]; /* Numele master-ului */
    char hostname[MAXPROC][NAMELEN]; /* Nume recepționate */

    int x[LENGTH][LENGTH]; /* Buffer-ul de transmisie */
    int y[LENGTH]; /* Buffer-ul de recepție */
```

```
int *sendcount, *displs; /* Pointeri pt. sendcounts și
deplasamente */
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &np);
MPI_Comm_rank(MPI_COMM_WORLD, &me);

gethostname(myname, NAMELEN);

/* Alocare memorie pentru sendcount și deplasamente */
sendcount = (int *)malloc(LENGTH*sizeof(int));
displs = (int *)malloc(LENGTH*sizeof(int));
/* Initializare sendcount și deplasamente */
for (i=0; i<LENGTH; i++) {
    sendcount[i] = i+1;
    displs[i] = i*LENGTH;
}
if (me == 0) { /* Proces 0 */
    /* Verificăm dacă sunt exact 8 procese */
    if (np != LENGTH) {
        printf("Trebuie sa utilizati %d procese\n", LENGTH);
        MPI_Finalize();
        exit(0);
    }
    /* Initializare matrice x cu valori 0 .. LENGTH*LENGTH-1 */
    for (i=0; i<LENGTH; i++) {
        for (j=0; j<LENGTH; j++)
            x[i][j] = i*LENGTH+j;
    }
    /* Afișare matrice înainte de distribuire */
    printf("Matricea initiala este\n");
    for (i=0; i<LENGTH; i++) {
        for (j=0; j<LENGTH; j++)
            printf("%4d ", x[i][j]);
        printf("\n");
    }
    printf("\nProcesul %d de pe masina %s distribuie matricea
la toate celelalte %d procese\n\n", me, myname, np);
    /* Distribuie elemente din matricea x către toate procesele,
prin plasare în matricea y */
    MPI_Scatterv(&x, sendcount, displs, MPI_INT, &y, LENGTH,
MPI_INT, root, MPI_COMM_WORLD);
    /* Procesul zero afișează elementele sale - de fapt elementul
său */
    for (i=0; i<sendcount[me]; i++)
        printf("%4d", y[i]);
    printf("
");
    printf(" de la procesul %d de pe statia %s\n", me,
myname);
    /* Primește mesaje ce conțin nume și date de la celelalte
processe */
    for (i=1; i<np; i++) {
```

```

    MPI_Recv (&hostname[i], NAMELEN, MPI_CHAR, i, nametag,
MPI_COMM_WORLD, &status);
    MPI_Recv (&y, LENGTH, MPI_INT, i, datatag,
MPI_COMM_WORLD, &status);
    MPI_Get_count(&status, MPI_INT, &count); /* primește nr
de elemente*/
    /* Afișează elementele primite*/
    for (j=0; j<count; j++)
        printf("%4d", y[j]);
    for (j=0; j<LENGTH-count; j++) /* Formateaza afisarea */
        printf(" ");
    printf(" de la procesul %d de pe calculatorul %s\n",
i, hostname[i]);
}
} else { /* celelalte procese execută codul ce urmează */
/* verificare */
if (np != LENGTH) { MPI_Finalize(); exit(0); }
/* plasare elemente primite de la proces 0 în matrice y */
MPI_Scatterv(&x, sendcount, displs, MPI_INT, &y, LENGTH,
MPI_INT, root, MPI_COMM_WORLD);
/* trimite numele înapoi procesului 0 */
MPI_Send (&myname, NAMELEN, MPI_CHAR, 0, nametag,
MPI_COMM_WORLD);
/* retrimite și elementele primite înapoi procesului 0 */
MPI_Send (&y, sendcount[me], MPI_INT, 0, datatag,
MPI_COMM_WORLD);
}
MPI_Finalize();
}

```

Pentru 8 procese și dimensiune matrice 8*8 avem rezultatele prezentate în figura 2.8.

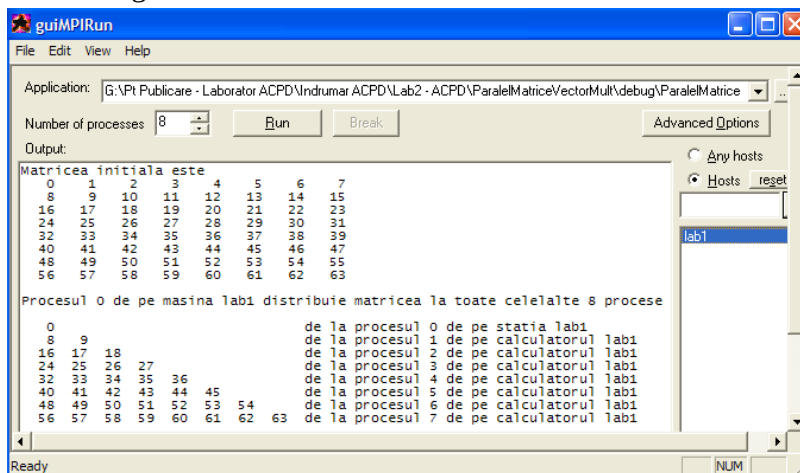


Figura 2.8 Verificare distribuție corectă date pentru 4 procese și dimensiune matrice egală cu 64

Desfășurarea lucrării:

1. Se va citi breviarul teoretic. Se atrage atenția asupra faptului că toate cunoștințele din această lucrare vor fi necesare și în derularea celorlalte lucrări.
2. Se vor studia problemele rezolvate (problemele prezentate pe parcursul acestui laborator), încercând găsirea altor posibilități de soluționare a acestora.
3. Programele prezentate în laborator (problemele rezolvate) vor fi scrise în *Visual C++* și se vor rula utilizând directivele din librăria *MPI*. Se va observa efectul rulării și buna lor execuție.
4. Se vor rezolva următoarele probleme propuse și se va urmări execuția lor corectă.
 - Să se implementeze un program MPI ce sumează numerele tuturor proceselor din comunicator prin conectarea acestora într-un inel, cu transferul numărului procesului și realizarea sumării pe măsură ce ating procesele (se va utiliza *MPI_Reduce*).
 - Implementați un program MPI prin care, folosind funcția *MPI_Scatter*, procesul 0 să distribuie elementele dintr-un vector de întregi, creat anterior în procesul cu id-ul 0, tuturor proceselor implicate. Procesele vor recepționa elementele și vor returna procesului 0 doua mesaje cu următorul conținut:
 - ❖ Numele mașinii pe care rulează
 - ❖ Elementele recepționate.

Procesul 0 va afișa informațiile primite de la celelalte procese.